

ERP Server

VPS Deployment Runbook — bare Ubuntu to running API

Target host	169.58.213.171
Operating system	Ubuntu 24.04.4 LTS (Noble Numbat)
Application	NestJS 11 · Prisma 6 · PostgreSQL 18
Public URL	https://169-58-213-171.sslip.io
Repository	github.com/vksoftnkl/ERP-Server
Branch deployed	DEV-v1
Document date	21 August 2026

1. What this document covers

A complete, reproducible procedure for deploying the ERP Server API onto a bare Ubuntu 24.04 virtual private server. Every command is run as `root` over SSH unless stated otherwise. Following it end to end on a fresh machine takes roughly 25–35 minutes, most of which is package installation and the initial database migration.

Two constraints that will break a naive deployment

This application *cannot* be deployed with a generic "install Node, run npm start" procedure. Two properties of the codebase force specific choices, and both fail in confusing ways if ignored:

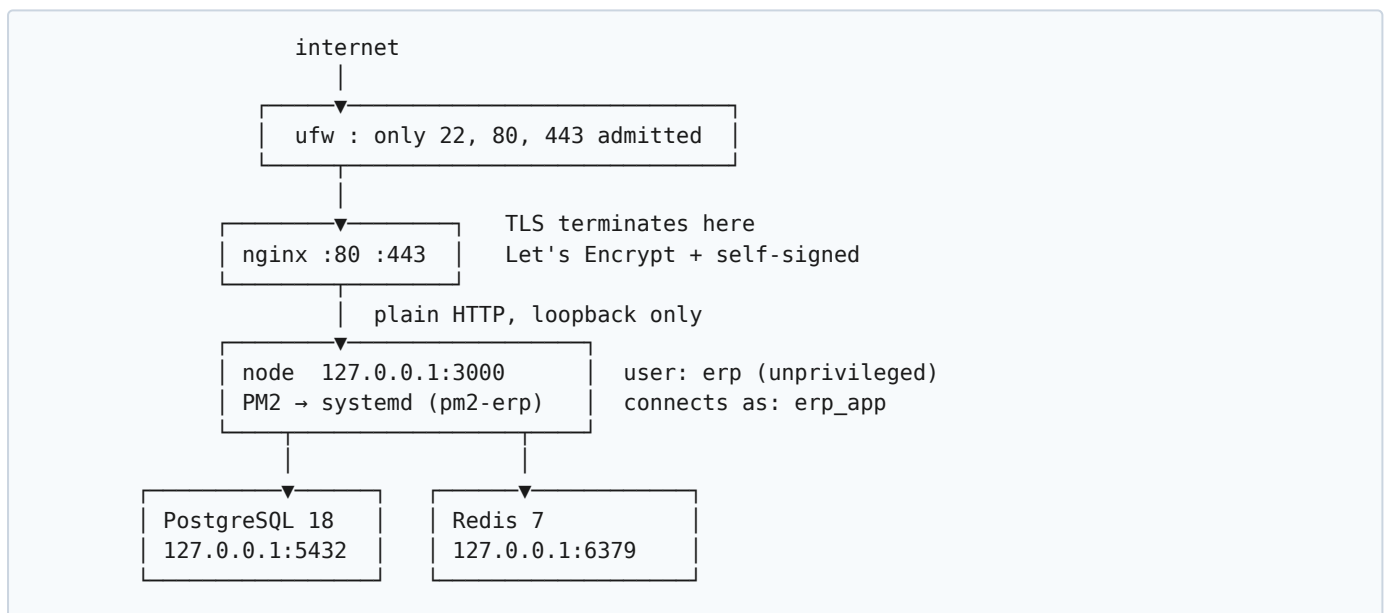
1. PostgreSQL 18 is mandatory.

The Prisma schema defaults columns to `uuidv7()` and no migration defines that function — it relies on the native PostgreSQL 18 builtin. Ubuntu 24.04 ships PostgreSQL 16, on which *every insert fails*. Section 5 therefore installs from the PGDG repository, not the distribution.

2. Migrations require a database superuser, but the application must not be one.

Seven migrations contain `ALTER TABLE ... OWNER to postgres`, which fails for any other role with `42501: must be able to SET ROLE "postgres"`. Development never reveals this because development connects as `postgres`. Sections 7 and 11 split the two roles apart.

Architecture



PostgreSQL, Redis and the Node process all bind the loopback interface only. nginx is the sole component reachable from the internet, and the firewall admits nothing else.

Contents

1. What this document covers
2. Prerequisites
3. Establish SSH access
4. Base packages and repositories
5. PostgreSQL 18, Node.js 22, nginx

- 6. Redis**
- 7. Database, roles and extensions**
- 8. Application user and source code**
- 9. Environment configuration**
- L0. Install dependencies and build**
- L1. Apply migrations and grant privileges**
- L2. Accounting-year partitions**
- L3. Run under PM2 and systemd**
- L4. nginx reverse proxy and TLS**
- L5. Firewall**
- L6. Verification**
- L7. Redeploying after a code change**
- L8. Troubleshooting**
- L9. Reference: files, ports and credentials**

2. Prerequisites

Requirement	Detail
VPS	Ubuntu 24.04 LTS, minimum 2 vCPU / 4 GB RAM / 20 GB disk. The reference host is 4 vCPU / 7.8 GB / 96 GB.
Root access	Password or SSH key for <code>root</code> . Section 3 replaces password login with key-based access.
Repository access	<code>github.com/vksoftnkl/ERP-Server</code> is public, so the server clones it without credentials. A private repository would need a deploy key.
Outbound network	The server must reach <code>apt.postgresql.org</code> , <code>deb.nodesource.com</code> , <code>registry.npmjs.org</code> , <code>github.com</code> and <code>acme-v02.api.letsencrypt.org</code> .
Inbound network	Ports 22, 80 and 443 must be reachable. Port 80 is required for Let's Encrypt HTTP-01 validation and for certificate renewal.

Conventions used below

Commands prefixed `$` run on *your workstation*; commands with no prefix run *on the VPS as root*. Placeholders in `ANGLE_BRACKETS` must be substituted. Real passwords are never printed in this document — they are generated on the server in step 7 and stored in `/root/.erp-deploy-secrets`.

3. Establish SSH access

Copy your public key to the server so every later step can run non-interactively.

```
# On your workstation. Creates a key only if you do not already have one.
$ ls ~/.ssh/id_ed25519.pub || ssh-keygen -t ed25519 -C "erp-deploy"

# Prompts once for the root password, then never again.
$ ssh-copy-id root@169.58.213.171

# Verify key-only login works.
$ ssh -o BatchMode=yes root@169.58.213.171 'echo CONNECTED; cat /etc/os-release | head -2'
```

Hardening (optional but recommended)

Once key login is confirmed, disable password authentication: `sed -i 's/^#\? PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config && systemctl reload ssh`. Do not do this before you have tested key login, or you will lock yourself out.

4. Base packages and repositories

Install the supporting tools, then register the two third-party APT repositories. PGDG supplies PostgreSQL 18 (Ubuntu only ships 16) and NodeSource supplies Node 22 LTS (Ubuntu ships 18, below the `engines.node >=20` floor in `package.json`).

```

export DEBIAN_FRONTEND=noninteractive NEEDRESTART_MODE=a

apt-get update -qq
apt-get install -y ca-certificates curl gnupg lsb-release ufw fail2ban \
    build-essential openssl rsync

# --- PGDG: PostgreSQL 18 ---
install -d /usr/share/postgresql-common/pgdg
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc \
    -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc
echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc] \
https://apt.postgresql.org/pub/repos/apt noble-pgdg main" \
    > /etc/apt/sources.list.d/pgdg.list

# --- NodeSource: Node.js 22 LTS ---
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key \
    | gpg --dearmor -o /usr/share/keyrings/nodesource.gpg
echo "deb [signed-by=/usr/share/keyrings/nodesource.gpg] \
https://deb.nodesource.com/node_22.x nodistro main" \
    > /etc/apt/sources.list.d/nodesource.list

apt-get update -qq

```

5. PostgreSQL 18, Node.js 22, nginx

```

apt-get install -y postgresql-18 postgresql-contrib-18 nodejs nginx

# Confirm the versions. PostgreSQL MUST report 18.x.
node --version      # v22.x
psql --version      # psql (PostgreSQL) 18.x
nginx -v            # nginx/1.24.0

# The decisive check: uuidv7() is a PostgreSQL 18 builtin.
# If this errors, you are on PG16 and every INSERT will fail later.
sudo -u postgres psql -tAc "select uuidv7()"

```

`postgresql-contrib-18` is required: it provides the `pgcrypto` and `btree_gist` extensions the migrations create.

6. Redis

Redis is not optional

`QueueModule` is imported unconditionally by `AppModule` and registers BullMQ queues that ignore the `REDIS_ENABLED` flag — that flag only gates the HTTP cache. Without a reachable Redis, BullMQ logs connection errors continuously.

```
apt-get install -y redis-server
```

```
# Loopback only -- never expose Redis to the network.
```

```
sed -i 's/^bind .*/bind 127.0.0.1 -::1/' /etc/redis/redis.conf
```

```
sed -i 's/^protected-mode .*/protected-mode yes/' /etc/redis/redis.conf
```

```
# BullMQ refuses to run against a Redis that may evict keys.
```

```
grep -q '^maxmemory-policy' /etc/redis/redis.conf \
```

```
&& sed -i 's/^maxmemory-policy .*/maxmemory-policy noeviction/' /etc/redis/redis.conf \
```

```
|| echo 'maxmemory-policy noeviction' >> /etc/redis/redis.conf
```

```
systemctl enable --now redis-server
```

```
systemctl restart redis-server
```

```
redis-cli ping
```

```
# PONG
```

```
redis-cli config get maxmemory-policy
```

```
# noeviction
```

7. Database, roles and extensions

Two roles are created. `erp_app` is what the running application uses and is deliberately **not** a superuser. `postgres` is used only to apply migrations. Keeping them apart matters: a SQL injection under a superuser role can reach `COPY ... PROGRAM`, and therefore a shell on the host.

```
# Generate strong secrets ON THE SERVER so they never transit a chat or terminal log.
SECRETS=/root/.erp-deploy-secrets
umask 077
{
  echo "DB_PASSWORD=$(openssl rand -base64 24 | tr -d '/+=' | head -c 32)"
  echo "JWT_SECRET=$(openssl rand -base64 48 | tr -d '/+=' | head -c 64)"
} > "$SECRETS"
chmod 600 "$SECRETS"
. "$SECRETS"

# Application role + database owned by postgres (migrations run as postgres).
sudo -u postgres psql -v ON_ERROR_STOP=1 <<SQL
DO \$\$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'erp_app') THEN
    CREATE ROLE erp_app LOGIN PASSWORD '${DB_PASSWORD}';
  ELSE
    ALTER ROLE erp_app LOGIN PASSWORD '${DB_PASSWORD}';
  END IF;
END
\$\$;
SQL
sudo -u postgres psql -c "CREATE DATABASE erp OWNER postgres ENCODING UTF8;"

# Pre-create the extensions as superuser. Migrations use IF NOT EXISTS,
# so this makes those statements no-ops.
sudo -u postgres psql -d erp -c \
  "CREATE EXTENSION IF NOT EXISTS pgcrypto; CREATE EXTENSION IF NOT EXISTS btree_gist;"

# Give the postgres role a password and record a migration-only URL.
PG_SU_PW=$(openssl rand -base64 24 | tr -d '/+=' | head -c 32)
sudo -u postgres psql -c "ALTER ROLE postgres LOGIN PASSWORD '${PG_SU_PW}';"
echo "PG_SUPERUSER_PASSWORD=${PG_SU_PW}" >> "$SECRETS"
echo "MIGRATE_DATABASE_URL=postgres://postgres:${PG_SU_PW}@127.0.0.1:5432/erp" >> "$SECRETS"
```

8. Application user and source code

The application runs as a dedicated unprivileged system user, never as root.

```
adduser --system --group --home /opt/erp-server --shell /bin/bash erp

install -d -o erp -g erp /opt/erp-server
sudo -u erp git clone --depth 1 --branch DEV-v1 \
  https://github.com/vksoftnkl/ERP-Server.git /opt/erp-server/app

install -d -o erp -g erp /opt/erp-server/app/logs
install -d -o erp -g erp /opt/erp-server/logs
install -d -m 750 /opt/erp-server/bin

sudo -u erp git -C /opt/erp-server/app log -1 --oneline
```

9. Environment configuration

`src/env.preload.ts` runs `dotenv.config()` before anything else, which reads `.env` relative to the process working directory. Because `dotenv` does **not** override variables already present in `process.env`, this file is the single source of truth only as long as nothing is pinned in the PM2 configuration.

```
. /root/.erp-deploy-secrets

cat > /opt/erp-server/app/.env <<ENVEOF
NODE_ENV=production
HOST=127.0.0.1           # loopback only; nginx is the sole client
PORT=3000
API_PREFIX=api
API_VERSION=1
REQUEST_BODY_LIMIT=10mb

# In production main.ts uses ONLY this list. An empty value means enableCors()
# never runs and every browser request is blocked.
CORS_ORIGINS=https://erp-frontend.vercel.app,https://169-58-213-171.sslip.io
CORS_CREDENTIALS=true

# TLS terminates at nginx; the app speaks plain HTTP on PORT.
HTTPS_ENABLED=false

DATABASE_URL=postgres://erp_app:${DB_PASSWORD}@127.0.0.1:5432/erp
DB_HOST=127.0.0.1
DB_PORT=5432
DB_USER=erp_app
DB_PASSWORD=${DB_PASSWORD}
DB_NAME=erp
DB_SSL=false

# Migrations are applied by the deploy script, not at boot.
DB_AUTO_MIGRATE=false
DB_AUTO_SEED=true
DB_SEED_FAIL_FAST=false

JWT_SECRET=${JWT_SECRET}
ACCESS_TOKEN_TTL_SECONDS=3600
REFRESH_TOKEN_TTL_SECONDS=604800

THROTTLE_ENABLED=true
THROTTLE_TTL=60
THROTTLE_LIMIT=10000

REDIS_ENABLED=true
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
REDIS_TTL=3600
REDIS_DB=0

LOG_FILE_PATH=logs/app.log
ERROR_LOG_FILE_PATH=logs/error.log
ENVEOF

chown erp:erp /opt/erp-server/app/.env
chmod 600 /opt/erp-server/app/.env
```


CORS is fail-closed in production

`main.ts` merges the localhost development defaults only when `NODE_ENV != production`. In production, an origin missing from `CORS_ORIGINS` is refused outright. Add every frontend origin here.

10. Install dependencies and build

`NODE_ENV` must be unset for the install

`@nestjs/cli` and `typescript` are `devDependencies`. With `NODE_ENV=production`, npm skips them and `npm run build` fails with "nest: not found". The `env -u NODE_ENV` below is deliberate.

```
cd /opt/erp-server/app

# postinstall regenerates the Prisma schema from prisma/ fragments,
# then runs `prisma generate`.
sudo -u erp env -u NODE_ENV HOME=/opt/erp-server \
  npm ci --include=dev --no-audit --fund=false

sudo -u erp env -u NODE_ENV HOME=/opt/erp-server npm run build

ls -l dist/src/main.js
```

11. Apply migrations and grant privileges

Migrations must run as `postgres`. Passing `DATABASE_URL` explicitly overrides the value in `.env`, because `dotenv` never overwrites a variable already set.

```
. /root/.erp-deploy-secrets
cd /opt/erp-server/app

sudo -u erp env -u NODE_ENV HOME=/opt/erp-server \
  DATABASE_URL="$MIGRATE_DATABASE_URL" \
  npm run prisma:migrate:deploy
```

Every object the migrations create is now owned by `postgres`, so the application role has no access to any of it. The following script repairs that and must be re-run after every migration. Save it as `/opt/erp-server/bin/grant-app-privileges.sql`:

```
DO
$$
DECLARE
    v_schema text;
BEGIN
    FOR v_schema IN
        SELECT nspname FROM pg_namespace
        WHERE nspname NOT LIKE 'pg\_%' AND nspname <> 'information_schema'
    LOOP
        -- CREATE, not just USAGE: the seed runner creates
        -- public.erp_seed_history on first boot.
        EXECUTE format('GRANT USAGE, CREATE ON SCHEMA %I TO erp_app', v_schema);
        EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES '
            'ON ALL TABLES IN SCHEMA %I TO erp_app', v_schema);
        EXECUTE format('GRANT USAGE, SELECT, UPDATE ON ALL SEQUENCES IN SCHEMA %I TO erp_app',
v_schema);
        EXECUTE format('GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA %I TO erp_app', v_schema);

        -- Objects a FUTURE migration creates as postgres are covered
        -- automatically, including new accounting-year partitions.
        EXECUTE format('ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA %I '
            'GRANT SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES ON TABLES TO
erp_app', v_schema);
        EXECUTE format('ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA %I '
            'GRANT USAGE, SELECT, UPDATE ON SEQUENCES TO erp_app', v_schema);
        EXECUTE format('ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA %I '
            'GRANT EXECUTE ON FUNCTIONS TO erp_app', v_schema);
    END LOOP;
END
$;
```

```
# /opt/erp-server/bin is mode 750 root:root, so the postgres OS user cannot
# read a path handed to psql -f. Pipe it in on stdin instead.
sudo -u postgres psql -d erp -v ON_ERROR_STOP=1 -f - \
    < /opt/erp-server/bin/grant-app-privileges.sql
```

12. Accounting-year partitions

Bill saves fail without this step

`sale_bill`, `sale_order`, `sale_quotation` and the accounting tables are LIST-partitioned by accounting year. A year with no partition makes every save return HTTP 500. The bootstrap block inside the partition migration reads `public.fiscal_years` — which is

empty after seeding

— so on a fresh install it creates nothing at all.

```
# Indian fiscal year runs April to March. Create the current and next year;
# add the previous one if you intend to import historical data.
for fy in 2025-2026 2026-2027 2027-2028; do
    sudo -u postgres psql -d erp -v ON_ERROR_STOP=1 \
        -c "SELECT public.ensure_acc_year_partitions('$fy'::char(9));"
done

# Confirm: each partitioned parent should list one child per year.
sudo -u postgres psql -d erp -c "
SELECT p.relname AS parent, count(*) AS partitions
FROM pg_class p
JOIN pg_inherits i ON i.inhparent = p.oid
WHERE p.relkind = 'p'
GROUP BY p.relname ORDER BY 1;"
```

The function is idempotent — re-running it for an existing year is a no-op.

13. Run under PM2 and systemd

The PM2 configuration lives **outside** the git working tree, because the redeploy script performs `git reset --hard` and would otherwise overwrite it. Its `env` block is intentionally empty so `.env` remains authoritative.

```

npm install -g pm2@latest

cat > /opt/erp-server/ecosystem.config.js <<'CFG'
module.exports = {
  apps: [{
    name: 'erp-api',
    script: 'dist/src/main.js',
    cwd: '/opt/erp-server/app',      // dotenv resolves .env relative to CWD
    instances: 1,
    exec_mode: 'fork',
    max_memory_restart: '1G',
    autorestart: true,
    max_restarts: 50,
    exponential_backoff_restart_delay: 1000, // 1s,2s,4s... not a hot loop
    min_uptime: '60s',              // seeds run before the port opens
    kill_timeout: 10000,
    error_file: '/opt/erp-server/logs/pm2-error.log',
    out_file:   '/opt/erp-server/logs/pm2-out.log',
    merge_logs: true,
    time: true,
  }],
};
CFG
chown erp:erp /opt/erp-server/ecosystem.config.js

```

Always `cd` before running `sudo -u erp`

The root shell starts in `/root`, which is mode `0700`. Spawning a process as `erp` from there fails with `spawn /usr/bin/node EACCES` — the child cannot access its inherited working directory. The error looks like a permissions problem with Node itself, which it is not.

```

cd /opt/erp-server
sudo -u erp HOME=/opt/erp-server pm2 start /opt/erp-server/ecosystem.config.js
sudo -u erp HOME=/opt/erp-server pm2 save

# Generate and enable the systemd unit that resurrects PM2 on boot.
pm2 startup systemd -u erp --hp /opt/erp-server
systemctl enable pm2-erp

# Order PM2 after its dependencies, or a reboot spends a minute crash-looping.
install -d /etc/systemd/system/pm2-erp.service.d
cat > /etc/systemd/system/pm2-erp.service.d/10-order.conf <<'UNIT'
[Unit]
After=postgresql.service redis-server.service
Wants=postgresql.service redis-server.service
UNIT
systemctl daemon-reload

# First boot runs the seeds before opening the port; allow ~30s.
curl -fsS http://127.0.0.1:3000/api/v1/health

```

14. nginx reverse proxy and TLS

Two TLS server blocks are configured. A real Let's Encrypt certificate serves the hostname that browsers use; a self-signed certificate serves the bare IP so `curl` and Postman still work. `sslip.io` is a public wildcard DNS service that resolves an IP encoded in the hostname, which makes a trusted certificate possible without buying a domain.

14.1 — Self-signed certificate for the bare IP

```
install -d /etc/nginx/ssl /var/www/html
openssl req -x509 -nodes -newkey rsa:2048 -days 3650 \
  -keyout /etc/nginx/ssl/erp-selfsigned.key \
  -out /etc/nginx/ssl/erp-selfsigned.crt \
  -subj "/CN=169.58.213.171" \
  -addext "subjectAltName=IP:169.58.213.171"
chmod 600 /etc/nginx/ssl/erp-selfsigned.key
```

14.2 — Shared proxy settings

Written to `/etc/nginx/snippets/erp-proxy.conf` and included by both server blocks so they cannot drift apart.

```
ssl_protocols      TLSv1.2 TLSv1.3;
ssl_ciphers        HIGH:!aNULL:!MD5;
ssl_session_cache  shared:SSL:10m;

client_max_body_size 20m;          # REQUEST_BODY_LIMIT is 10mb; leave margin
proxy_connect_timeout 10s;
proxy_send_timeout   120s;
proxy_read_timeout   120s;        # large grid/report queries run long

access_log /var/log/nginx/erp-access.log;
error_log  /var/log/nginx/erp-error.log;

location / {
    proxy_pass http://erp_api;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme; # else Swagger builds http:// URLs
    proxy_set_header X-Forwarded-Host $host;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection $connection_upgrade;
    proxy_buffering off;
}
```

14.3 — Site configuration

Written to `/etc/nginx/sites-available/erp`:

```

upstream erp_api { server 127.0.0.1:3000; keepalive 32; }
map $http_upgrade $connection_upgrade { default upgrade; '' close; }

server {
    listen 80 default_server;
    server_name _;
    # Must stay on plain HTTP: certbot renews through this path.
    location /.well-known/acme-challenge/ { root /var/www/html; }
    location / { return 301 https://$host$request_uri; }
}

server {                                     # trusted cert -- use this from browsers
    listen 443 ssl http2;
    server_name 169-58-213-171.sslip.io;
    ssl_certificate      /etc/letsencrypt/live/169-58-213-171.sslip.io/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/169-58-213-171.sslip.io/privkey.pem;
    include /etc/nginx/snippets/erp-proxy.conf;
}

server {                                     # self-signed fallback for the bare IP
    listen 443 ssl http2 default_server;
    server_name _;
    ssl_certificate      /etc/nginx/ssl/erp-selfsigned.crt;
    ssl_certificate_key  /etc/nginx/ssl/erp-selfsigned.key;
    include /etc/nginx/snippets/erp-proxy.conf;
}

```

nginx 1.24 syntax

Ubuntu 24.04 ships nginx 1.24, which predates the standalone `http2 on;` directive introduced in 1.25.1. Use `listen 443 ssl http2;` instead, or the configuration test fails with `unknown directive "http2"`.

14.4 — Enable the site and issue the certificate

```

ln -sf /etc/nginx/sites-available/erp /etc/nginx/sites-enabled/erp
rm -f /etc/nginx/sites-enabled/default
nginx -t && systemctl reload nginx && systemctl enable nginx

apt-get install -y certbot
certbot certonly --webroot -w /var/www/html \
    -d 169-58-213-171.sslip.io \
    --non-interactive --agree-tos --register-unsafely-without-email

# certonly does NOT reload nginx after a renewal, so a renewed certificate
# would sit unused on disk. This hook fixes that.
install -d /etc/letsencrypt/renewal-hooks/deploy
printf '#!/bin/sh\nsystemctl reload nginx\n' \
    > /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh
chmod 750 /etc/letsencrypt/renewal-hooks/deploy/reload-nginx.sh

systemctl reload nginx
certbot renew --dry-run          # must report "all simulated renewals succeeded"

```

15. Firewall

Allow port 22 before enabling ufw

Enabling the firewall without an SSH rule terminates your session and locks you out of the server permanently unless you have console access.

```
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw default deny incoming
ufw default allow outgoing
ufw --force enable
systemctl enable ufw

ufw status verbose

# 3000, 5432 and 6379 must appear as 127.0.0.1 only.
ss -tlnp | grep -E ':(3000|5432|6379) '
```

16. Verification

Run these from your workstation. Every line must match the expected column.

Check	Expected
<code>curl https://169-58-213-171.sslip.io/api/v1/health</code>	200, <code>status:"ok"</code> , database and cache both up
<code>curl -o /dev/null -w '%{http_code}' http://169.58.213.171/</code>	301 redirect to HTTPS
<code>curl https://169-58-213-171.sslip.io/api/docs</code>	200 — Swagger UI
<code>curl http://169.58.213.171:3000/</code>	connection refused / timeout
<code>nc -z 169.58.213.171 5432</code>	refused — database not exposed

And on the server:

```
# All five must report active + enabled.
for s in postgresql redis-server nginx pm2-erp ufw; do
  printf '%-16s %s %s\n' "$s" "$(systemctl is-active $s)" "$(systemctl is-enabled $s)"
done

# Process should be online, running as erp, with a low restart count.
cd /opt/erp-server && sudo -u erp HOME=/opt/erp-server pm2 list

# Seeds: expect every row to read "success".
sudo -u postgres psql -d erp -c \
  "SELECT last_status, count(*) FROM public._erp_seed_history GROUP BY 1;"

# Migrations: expect 0 unfinished.
sudo -u postgres psql -d erp -c \
  "SELECT count(*) AS applied,
    count(*) FILTER (WHERE finished_at IS NULL) AS unfinished
  FROM _prisma_migrations;"

# erp_app must NOT be a superuser.
sudo -u postgres psql -c \
  "SELECT rolname, rolsuper FROM pg_roles WHERE rolname IN ('erp_app','postgres');"
```

A successful first deployment reports 285 migrations applied with none unfinished, 22 seeds succeeded, and roughly 157 tables visible to `erp_app`.

17. Redeploying after a code change

Deployment is manual by design

Pushing to GitHub does

not

update the server. The redeploy script applies database migrations, so it should run while someone is watching rather than on every push. The full cycle is

commit → push → run the deploy script

. The script fetches from GitHub, so an unpushed commit is invisible to it.


```
$ git push # updates GitHub only
$ ssh root@169.58.213.171 /opt/erp-server/bin/deploy.sh # updates the live server
```

`deploy.sh` performs, in order, aborting on the first failure:

1. `git fetch` and `git reset --hard origin/<branch>`
2. `npm ci --include=dev` — with `NODE_ENV` unset
3. `npm run build`
4. `prisma migrate deploy` using the superuser URL
5. re-run `grant-app-privileges.sql`
6. `ensure_acc_year_partitions()` for the current and next fiscal year
7. `pm2 reload erp-api` and `pm2 save`
8. poll `/api/v1/health`; on failure print the logs and the rollback command

Because the build and migrations run *before* PM2 is touched, a broken commit never replaces a working process. To roll back, reset to the previous commit and redeploy:

```
sudo -u erp git -C /opt/erp-server/app reset --hard <PREVIOUS_SHA>
/opt/erp-server/bin/deploy.sh
```

Never edit files inside `/opt/erp-server/app`

The deploy script runs `git reset --hard`, which destroys local edits. The two files deliberately kept outside that directory — `/opt/erp-server/app/.env` (gitignored) and `/opt/erp-server/ecosystem.config.js` — survive.

18. Troubleshooting

Symptom	Cause and remedy
ERROR: must be able to SET ROLE "postgres" (42501)	Migrations were run as <code>erp_app</code> . Seven migrations contain <code>ALTER TABLE ... OWNER to postgres</code> . Re-run with <code>DATABASE_URL="\$MIGRATE_DATABASE_URL"</code> . If a migration is left half-applied on a fresh database, drop and recreate it rather than repairing by hand.
function uuidv7() does not exist	PostgreSQL 16 is installed. <code>uuidv7()</code> is a PostgreSQL 18 builtin and no migration defines it. Install <code>postgresql-18</code> from PGDG (section 5).
spawn /usr/bin/node EACCES	A process was started as <code>erp</code> from <code>/root</code> , which is mode 0700, so the child cannot access its working directory. <code>cd /opt/erp-server</code> first.
nest: not found	<code>npm ci</code> ran with <code>NODE_ENV=production</code> , so devDependencies were skipped. Re-run with <code>env -u NODE_ENV</code> .
nginx: unknown directive "http2"	nginx 1.24 predates the standalone directive. Use <code>listen 443 ssl http2;</code> .
HTTP 500 when saving a bill or order	No partition exists for that accounting year. Run <code>SELECT public.ensure_acc_year_partitions('YYYY-YYYY');</code> . Note <code>fiscal_years</code> is empty on a fresh install, so the automatic bootstrap creates nothing.
permission denied for table <name>	A migration created an object owned by <code>postgres</code> and the grants were not re-applied. Re-run <code>grant-app-privileges.sql</code> . Add any missing grant to that file — never repoint the application at the <code>postgres</code> role.
CORS error in the browser	In production <code>main.ts</code> uses only <code>CORS_ORIGINS</code> ; the localhost defaults are not merged in. Add the origin, then <code>pm2 reload erp-api --update-env</code> .
Frontend cannot reach the API although curl works	The frontend is calling the bare IP, which carries only a self-signed certificate. Browsers reject it for cross-origin requests with no click-through. Use <code>https://169-58-213-171.sslip.io</code> .
ECONNREFUSED 127.0.0.1:6379	Redis is down. BullMQ needs it regardless of <code>REDIS_ENABLED</code> , which gates only the HTTP cache. <code>systemctl status redis-server</code> .
Repeated restarts right after a reboot	PM2 started before PostgreSQL accepted connections. Install the <code>10-order.conf</code> drop-in from section 13.

Useful commands

```
cd /opt/erp-server && sudo -u erp HOME=/opt/erp-server pm2 logs erp-api --lines 100
cd /opt/erp-server && sudo -u erp HOME=/opt/erp-server pm2 restart erp-api
tail -f /var/log/nginx/erp-error.log
journalctl -u pm2-erp -n 50 --no-pager
sudo -u postgres psql -d erp
```

19. Reference

Filesystem

Path	Contents
/opt/erp-server/app	Git working tree. Reset on every deploy.
/opt/erp-server/app/.env	Configuration, mode 600. Gitignored, survives deploys.
/opt/erp-server/ecosystem.config.js	PM2 configuration. Outside the git tree on purpose.
/opt/erp-server/bin/	deploy.sh , grant-app-privileges.sql , verify-db.sh
/opt/erp-server/logs/	PM2 stdout and stderr
/root/.erp-deploy-secrets	DB password, JWT secret, superuser URL. Mode 600.
/etc/nginx/sites-available/erp	Reverse proxy and TLS configuration
/etc/letsencrypt/live/...	Trusted certificate and key

Ports

Port	Bound to	Service
22	0.0.0.0	SSH — open
80	0.0.0.0	nginx — redirects to HTTPS, serves ACME challenges
443	0.0.0.0	nginx — TLS termination
3000	127.0.0.1	Node application — not reachable externally
5432	127.0.0.1	PostgreSQL 18 — not reachable externally
6379	127.0.0.1	Redis 7 — not reachable externally

Database access from a workstation

PostgreSQL is firewalled and loopback-bound, so there is no direct external connection. Tunnel over SSH instead:

```
$ ssh -N -L 55432:127.0.0.1:5432 root@169.58.213.171
$ psql "postgres://erp_app:<DB_PASSWORD>@127.0.0.1:55432/erp"
```

In pgAdmin, use the **SSH Tunnel** tab rather than pasting a URL into the host field: tunnel host 169.58.213.171 , user root , identity file; then connection host 127.0.0.1 , port 5432 , database erp , user erp_app . If pgAdmin was installed as a snap it cannot read ~/.ssh — snaps are denied hidden directories — so place the key in a non-hidden folder such as ~/pgadmin-ssh/ .

Credentials

Passwords are intentionally omitted from this document

They are generated on the server during step 7 and stored in `/root/.erp-deploy-secrets` (mode 600). Read them with `ssh root@169.58.213.171 'cat /root/.erp-deploy-secrets'`. Never commit them, and keep `.env` and every `.env.*` variant out of version control.